

ER SCHEMA RELATIONSHIPS

Entity	Key Attribute	Important Attributes
User	user_id	full_name, email, phone, date_of_birth, home_city, account_status
Vendor	vendor_id	company_name, vendor_type, gst_number, contact_name, contact_email
TransportRoute	route_id	transport_mode, origin_city, destination_city, vehicle_number
RouteSchedule	schedule_id	departure_time, arrival_time, days_of_week, is_active
ScheduleClass	class_id (<i>partial key</i>)	class_name, total_seats, base_fare
ScheduleDeparture	departure_id (<i>partial key</i>)	departure_date, available_seats, dynamic_fare, status
Hotel	hotel_id	hotel_name, city, star_rating, checkin_time, checkout_time
RoomCategory	room_category_id (<i>partial key</i>)	category_name, max_guests, price_per_night, total_rooms
RoomAvailability	(<i>partial key:</i> room_category_id + available_date)	available_rooms
HolidayPackage	package_id	package_name, theme, duration_days, base_price_per_person
PackageDeparture	package_departure_id (<i>partial key</i>)	departure_date, available_seats, price_per_person, status
PromoCode	promo_id	code, discount_type, discount_value, valid_from, valid_to, applicable_on
Passenger	passenger_id (<i>partial key</i>)	full_name, age, gender, govt_id_type, govt_id_number, seat_number

Complete Relationship Summary

#	Relationship	Entities	Cardinality	Ternary?
1	operates	Vendor → TransportRoute	1:N	No
2	has_schedule	TransportRoute → RouteSchedule	1:N	No
3	has_class	RouteSchedule → ScheduleClass	1:N	No
4	generates	RouteSchedule + ScheduleClass → ScheduleDeparture	1:N:M	Yes
5	lists	Vendor → Hotel	1:N	No
6	has_rooms	Hotel → RoomCategory	1:N	No
7	tracked_on	RoomCategory → RoomAvailability	1:N	No
8	offers	Vendor → HolidayPackage	1:N	No
9	has_departure	HolidayPackage → PackageDeparture	1:N	No
10	books_transport	User + ScheduleDeparture → Booking	M:N	Yes
11	books_hotel	User + RoomCategory → Booking	M:N	Yes
12	books_package	User + PackageDeparture → Booking	M:N	Yes
13	applies	PromoCode → Booking	1:N	No
14	writes	User → Review	1:N	No
15	has_passenger	TransportBooking → Passenger	1:N	No

RELATIONSHIP 1 — operates

Between: Vendor → TransportRoute

A vendor operates transport routes.

- One Vendor operates MANY TransportRoutes → **1:N**
- Vendor participation: **Partial** (single line) — a vendor can exist without having listed any route yet
- TransportRoute participation: **Total** (double line) — a route MUST belong to some vendor, it cannot exist without one

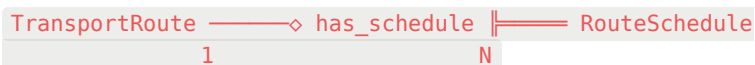


◆ RELATIONSHIP 2 — has_schedule

Between: TransportRoute → RouteSchedule

A route has timing schedules (6AM flight, 9PM flight etc.)

- One TransportRoute has MANY RouteSchedules → **1:N**
- TransportRoute participation: **Partial** (a route can exist with no schedule defined yet)
- RouteSchedule participation: **Total** (a schedule MUST belong to a route)



◆ RELATIONSHIP 3 — has_class

Between: RouteSchedule → ScheduleClass

Every schedule has seat classes (Economy, Business etc.)

- One RouteSchedule has MANY ScheduleClasses → **1:N**
- RouteSchedule participation: **Partial**
- ScheduleClass participation: **Total** (double line — weak entity, must have parent)



◆ RELATIONSHIP 4 — generates

Between: RouteSchedule + ScheduleClass → ScheduleDeparture

⚠ **This is your first TERNARY relationship.** Your assignment explicitly says to use ternary relationships where appropriate — this is the perfect place.

Why ternary? Because a `ScheduleDeparture` (one real bookable slot) is defined by THREE things simultaneously:

- WHICH schedule (6AM Mumbai → Delhi)
- WHICH class (Economy)
- WHICH date (15th March)

All three together create one departure slot. You cannot define it with just two of them. This is genuinely ternary — not forced.

- RouteSchedule : ScheduleDeparture → **1:N** (one schedule generates many departures)
- ScheduleClass : ScheduleDeparture → **1:N** (one class appears in many departures)
- ScheduleDeparture participation: **Total** on both sides (double line — weak entity)



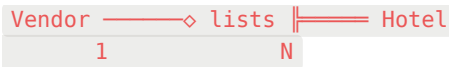
In Dia: draw the diamond in the centre, three lines coming out — one to RouteSchedule, one to ScheduleClass, one to ScheduleDeparture. Label cardinality on each line.

◆ RELATIONSHIP 5 — lists

Between: Vendor → Hotel

A vendor lists hotel properties on the platform.

- One Vendor lists MANY Hotels → **1:N**
- Vendor participation: **Partial**
- Hotel participation: **Total** (a hotel must belong to a vendor)

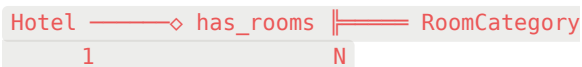


◆ RELATIONSHIP 6 — **has_rooms**

Between: Hotel → RoomCategory

A hotel has multiple room categories.

- One Hotel has MANY RoomCategories → **1:N**
- Hotel participation: **Partial**
- RoomCategory participation: **Total** (weak — must have hotel)

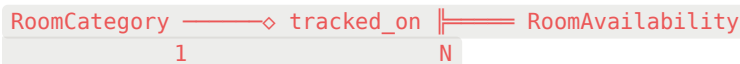


◆ RELATIONSHIP 7 — **tracked_on**

Between: RoomCategory → RoomAvailability

For each room category, availability is tracked date by date.

- One RoomCategory has MANY RoomAvailability records → **1:N**
- RoomCategory participation: **Partial**
- RoomAvailability participation: **Total** (weak — meaningless without room category)



◆ RELATIONSHIP 8 — **offers**

Between: Vendor → HolidayPackage

A vendor creates and offers holiday packages.

- One Vendor offers MANY HolidayPackages → **1:N**
- Vendor participation: **Partial**
- HolidayPackage participation: **Total**

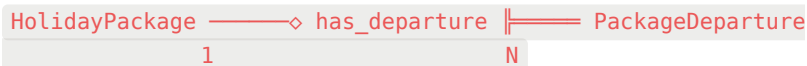


◆ RELATIONSHIP 9 — **has_departure**

Between: HolidayPackage → PackageDeparture

A package runs on multiple fixed departure dates.

- One HolidayPackage has MANY PackageDepartures → **1:N**
- HolidayPackage participation: **Partial**
- PackageDeparture participation: **Total** (weak entity)



◆ RELATIONSHIP 10 — **books_transport**

Between: User + ScheduleDeparture → (Booking)

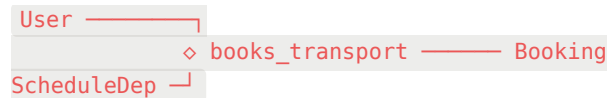
⚠ **This is your second TERNARY — and the most important one.**

This is also **aggregation** territory from your Image 2. When a user books a transport seat:

- A specific **User** is involved
- A specific **ScheduleDeparture** (which already encodes route + schedule + class + date) is involved
- The act of booking produces a **Booking** record with payment info, status, amount

The relationship **books_transport** connects User and ScheduleDeparture, and the Booking record is the result of that relationship — carrying its own attributes (amount, payment, status). Because Booking carries data, it becomes a **relationship entity** (drawn as a diamond inside a rectangle in formal ER).

- User : Booking → **1:N** (one user makes many bookings)
- ScheduleDeparture : Booking → **1:N** (one departure slot can be booked many times by different users)
- User participation: **Partial** (user may have no bookings)
- ScheduleDeparture participation: **Partial** (a departure may have no bookings yet)



Under `books_transport` (TransportBooking), Passenger hangs as a weak entity:



◆ RELATIONSHIP 11 — `books_hotel`

Between: User + RoomCategory → (Booking)

Same pattern. User books a specific RoomCategory at a Hotel for a date range.

- User : Booking → **1:N**
- RoomCategory : Booking → **1:N**
- Both sides: **Partial** participation

◆ RELATIONSHIP 12 — `books_package`

Between: User + PackageDeparture → (Booking)

User books a specific PackageDeparture slot.

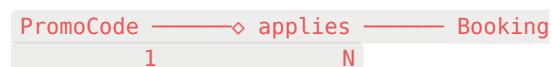
- User : Booking → **1:N**
- PackageDeparture : Booking → **1:N**
- Both sides: **Partial** participation

◆ RELATIONSHIP 13 — `applies`

Between: PromoCode → Booking

A promo code is applied to a booking.

- One PromoCode applies to MANY Bookings → **1:N**
- PromoCode participation: **Partial** (a promo may never be used)
- Booking participation: **Partial** (a booking may have no promo)



◆ RELATIONSHIP 14 — **writes**

Between: User → Review (on Hotel / Package / Route)

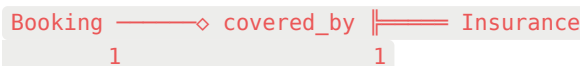
- One User writes MANY Reviews → **1:N**
- User participation: **Partial**
- Review participation: **Total** (a review must have a writer)

◆ RELATIONSHIP 15 — **covered_by**

Between: Booking → Insurance

A booking can optionally have travel insurance added to it.

- One Booking has at most ONE Insurance record → **1:1**
- Booking participation: **Partial** (most bookings won't have insurance)
- Insurance participation: **Total** (an insurance record must belong to a booking)



Insurance attributes:

```
insurance_id
provider_name
plan_name
plan_type      (SINGLE_TRIP / MULTI_TRIP)
coverage_summary
premium_amount
valid_from
valid_to
claim_status   (NOT_CLAIMED / CLAIM_FILED / SETTLED)
```

◆ RELATIONSHIP 16 — **cancelled_as**

Between: Booking → CancellationRefund

When a booking is cancelled, one refund record is created.

- One Booking has at most ONE CancellationRefund → **1:1**
- Booking participation: **Partial** (most bookings are never cancelled)
- CancellationRefund participation: **Total** (a refund record must have a booking)



CancellationRefund attributes:

```
cancellation_id
cancelled_at
reason
refund_amount
refund_status (PENDING / PROCESSED / REJECTED)
processed_at
```

◆ RELATIONSHIP 17 — reviewed_by

Between: Booking → Review

After a booking is completed, the user reviews the service they experienced. The review connects to Booking — not directly to User — because this enforces that only people who actually booked can review.

- One Booking has at most ONE Review → **1:1**
- Booking participation: **Partial** (most completed bookings won't get reviewed)
- Review participation: **Total** (a review must have a booking behind it)

```
Booking —◇ reviewed_by ||==== Review
      1                1
```

Review attributes:

```
review_id
entity_type (TRANSPORT / HOTEL / PACKAGE)
entity_id
star_rating (1-5)
title
comment
reviewed_at
```

◆ RELATIONSHIP 18 — DemandTag (standalone, no FK)

DemandTag is special. It's a computed cache — a background job looks at

```
ScheduleDeparture.available_seats , RoomAvailability.available_rooms , and
PackageDeparture.available_seats
```

 and writes a tag.

It has **no hard foreign key** to anything. It's a loose reference. In ER terms it's a standalone weak entity that references service entities by type+id polymorphically.

In your ER diagram — draw `DemandTag` as a standalone rectangle with a **dashed border** or annotate it as *"computed — no FK constraint"*. Don't draw a relationship

line from it to anything. Just place it with a note.

```
[ - - - - - ]  
DemandTag  
| (computed cache)|  
- - - - -
```

DemandTag attributes:

```
tag_id  
service_type      (TRANSPORT / HOTEL / PACKAGE)  
service_id        (loose reference – no FK)  
tag_value          (HIGH_DEMAND / FILLING_FAST / LOW_AVAILABILITY)  
last_computed
```